Emergency Training Simulator POC Report

Prepared as a project progress report for Dr. Cameron.

Executive summary

The long-term project vision is an AI-driven, real-time adaptive emergency-response training platform for firefighters and EMTs. The full system points toward agentic AI scenario generation, immersive training rooms, physical mannequins, multimodal sensing, structured verification and validation, scalable hub-and-spoke deployment, and post-scenario review.

This repository is not the full platform yet. It is the first proof-of-concept foundation underneath that larger direction.

The current POC proves a smaller but important loop: a manual or scripted trainee action reaches the backend, changes deterministic scenario state, gets checked by a validation layer, returns client-ready state, writes a JSONL log, and can be replayed and reviewed afterward.

Full project north star

The full project eventually aims to support a closed-loop training experience:

```
trainee speaks, moves, and performs procedures
-> sensing captures behavior
-> AI interprets actions
-> scenario state adapts
-> validation checks safety and coherence
-> immersive environment updates
-> session data is logged
-> review supports learning and system improvement
```

Future capabilities include AI and agentic scenario adaptation, multimodal sensing, projection mapping, mannequin integration, cloud services, distributed training sites, and validation against structured clinical or training references.

Those are future full-system goals. They are not current repo capabilities.

Current POC scope

The current repo focuses on one trusted backend loop:

```
manual/stubbed trainee action
-> FastAPI backend receives event
-> deterministic scenario engine updates state
-> POC validation checks rule conformance
-> WebSocket client receives state
-> JSONL log is written
-> replay and review explain what happened
```

This scope is intentionally narrow. The goal is to make the core logic testable, repeatable, and explainable before adding AI, Unreal, sensing, projection, or real physics.

Why backend-first

It would be tempting to start with Unreal, generated assets, AI calls, sensors, or projection mapping because those pieces look more impressive.

The risk is that those layers can hide an unclear source of truth.

The backend comes first because the full system will eventually need to answer basic questions reliably:

- What action was attempted?
- What state changed?
- What was rejected or warned?
- What did the scenario look like at the end?
- Can the session be replayed?
- Can an instructor understand what happened afterward?

If those answers are not clear, visuals and AI will not fix the foundation.

Current implementation

The current implementation includes:

- FastAPI health endpoint
- WebSocket session endpoint
- deterministic YAML scenario engine
- Pydantic message models
- JSON schemas and examples
- POC validation rules
- local JSONL session logs
- replay script
- improved post-scenario review summary
- one-command local demo
- future Unreal client contract
- text-only Unreal display preview
- environment object contract
- basic security guardrails
- lightweight GitHub Actions CI

The demo scenario is a small smoke-room POC. It includes a wooden chair and an entry door as environment cues. The chair can block the path. The door can be open or closed. The happy path moves the chair, opens the door, enters the room, and removes the occupant. The warning/error path shows what happens when the trainee ignores sequence or environmental cues.

This scenario is a software fixture only. It is not emergency guidance.

Review and replay as the proof point

The strongest result so far is not just that the demo runs.

The strongest result is that the system can explain what happened.

The review summary now includes:

- session basics
- final phase and outcome
- end condition status
- validation counts
- replay verification
- action timeline
- environment cues
- warnings

- errors
- key observations
- reviewer takeaways

For example, the warning/error path can show that a trainee attempted actions too early, that the entry door remained closed, that warnings/errors were recorded, and that replay verified the ordered events.

This matters because training systems need debriefing. A visually impressive simulation is less useful if it cannot explain the session afterward.

Validation boundary

Current validation means POC rule conformance only.

It answers:

Did this action follow this toy scenario's deterministic rules?

It does not claim:

- clinical validity
- medical correctness
- official EMT protocol authority
- firefighter protocol authority
- certification scoring
- real-world operational guidance

The full project may eventually include a structured verification and validation framework trained or checked against validated references. That work belongs to a later stage.

Security and trust boundary

The backend is the source of truth.

A client can request an action, but the backend decides:

- current scenario state
- validation status
- warnings and errors
- final outcome
- replay/review output

The current repo includes basic POC guardrails:

- safe identifier checks
- safe scenario/action names
- small payload limits
- sequence checks
- duplicate event rejection
- max active sessions
- max actions per session
- clearer WebSocket error codes

This is not production security. There is no authentication, authorization, TLS, cloud hardening, real rate limiting, or real data policy yet.

Relationship to future Unreal work

Unreal is treated as a future client, not the source of truth.

The future Unreal client should display backend-owned state and send trainee actions. It should not decide validation results, final outcomes, or review summaries.

The current repo includes a text-only future Unreal contract and display preview. It does not build a full Unreal scene.

Relationship to future assets and physics

The current chair and door are scenario objects and metadata, not generated 3D assets.

Future generated assets may come from tools like Hunyuan 3D or similar pipelines, but assets should be reviewed manually before use. Blender is likely useful later as the review and cleanup tool before Unreal import.

Physics is also future work. The current repo can represent physics preview metadata, but it does not simulate gravity, collision, tipping, mass, or force.

Two-track roadmap

The project now has two tracks.

POC track:

- keep the deterministic backend loop stable
- improve replay and review
- refine message contracts
- add small deterministic scenarios
- strengthen UX and instructor review flow
- prepare for a thin Unreal display client

Full-system track:

- AI scenario generation and adaptation
- validation framework expansion
- multimodal sensing
- immersive rooms and projection mapping
- mannequin integration
- real physics and environment simulation
- hub-and-spoke deployment
- cloud services and production security

The POC track should continue to move first. It creates the foundation that the full-system track will need.

Recommended next steps

The next useful work should stay close to the proof point:

1. improve instructor/reviewer examples
2. add a small evaluation rubric for Dr. Cameron
3. strengthen the future Unreal text debug contract
4. add another deterministic toy scenario
5. keep the backend as source of truth

Unreal, generated assets, AI calls, sensing, and deployment should wait until the contracts and review loop are stable.

Closing

This repo is small on purpose.

It does not try to be the final adaptive training platform yet. It proves the first trustworthy backend loop: action in, state update, validation result, client message, log, replay, and review.

That foundation is what makes the larger project believable.